

ACTC - Homework 3

In your final ACTC assignment you will build a simple container orchestration system implementing the core functionality of Kubernetes.

Though in production we might choose a language like Rust (for correctness, maintainability and the great ecosystem) or Go (for the strong cloud-specific ecosystem which means most developers with experience in cloud technologies are fluent), for our prototype we will choose Python (for speed of implementation vs C, and because you all learned it, which cannot be said of Rust or Go).

Often, when we embark on a difficult technical project, we start by building a proof of concept (POC) - just the hard thing without all the stuff around it that makes it usable in the real world - to ensure that we know what hardships we will encounter and that our design is sound before we commit to it. Of course, some challenges may not come up in the POC, but the more we learn and derisk upfront, the better.

To put your time and effort to best didactic use, you won't start from scratch, you'll start from a POC, and you will develop it into a fully usable MVP (minimum viable product, the first version of a product that someone could actually get value from).

Read this assignment description carefully till the end and start working on the assignment from the day it is published.

What you're building

- A tool to orchestrate `podman` containers.
- Using Python with the `uv` package manager, with `fastapi` for the REST API, `subprocess` for running podman commands, and `pytest` for tests (you may need to read up on these libraries, they have great documentation).
- All in a single process, runnable as `uv run orchestrator.py`.
- The design is based on Kubernetes, with familiar resources like Pods, Containers, ReplicaSets, Services (with some simplifications, see "What you're not building").
- All configuration is done by creating, updating and deleting *resources* through a REST API that is a subset of the Kubernetes REST API.
- For example, these calls create a `health-replicaset` *ReplicaSet* that runs 3 replicas of a container based on the `health` image, a `health` *Service* that listens on port 2000 and load balances between those replicas, and a `ping` *Container* based on the `ping` image that has a `HEALTH_SERVICE` environment variable pointing at the `health` service:

Shell

```
export ORCHESTRATOR=http://localhost:3000
```

```
curl -X POST
```

```
https://$ORCHESTRATOR/api/apps/v1/namespaces/default/replicasets
```

```
\
```

```
-H "Content-Type: application/json" \  
-d '{  
  "apiVersion": "apps/v1",  
  "kind": "ReplicaSet",  
  "metadata": {  
    "name": "health-replicaset"  
  },  
  "spec": {  
    "replicas": 3,  
    "selector": {  
      "name": "health"  
    },  
    "template": {  
      "metadata": {  
        "app": "health"  
      },  
      "spec": {  
        "containers": [{  
          "name": "health",  
          "image": "health:latest"  
        }]  
      },  
    },  
  },  
}'
```

```
curl -X POST https://$ORCHESTRATOR/api/v1/namespaces/default/pods
```

```
\
```

```
-H "Content-Type: application/json" \  
-d '{  
  "apiVersion": "v1",
```

```

    "kind": "Service",
    "metadata": {
      "name": "health-service"
    },
    "spec": {
      "selector": {
        "app": "health"
      },
      "ports": [{
        "protocol": "TCP",
        "port": 2000,
        "targetPort": 5000
      }],
      "type": "ClusterIP"
    }
  }
}'

```

```

curl -X POST https://$ORCHESTRATOR/api/v1/namespaces/default/pods \
\
-H "Content-Type: application/json" \
-d '{
  "apiVersion": "v1",
  "kind": "Pod",
  "metadata": {
    "name": "ping"
  },
  "spec": {
    "containers": [{
      "name": "ping",
      "image": "ping:latest",
      "env": {
        "HEALTH_SERVICE": "health-service:2000"
      }
    }]
  }
}'

```

- If we then ran this, it would cause 2 additional replicas to be created:

Shell

```
curl -X PUT
https://$ORCHESTRATOR/api/apps/v1/namespaces/default/replicaset/
health-replicaset \
  -H "Content-Type: application/json" \
  -d '{
    "apiVersion": "apps/v1",
    "kind": "ReplicaSet",
    "metadata": {
      "name": "health-replicaset"
    },
    "spec": {
      "replicas": 5,
      "selector": {
        "name": "health"
      },
      "template": {
        "metadata": {
          "app": "health"
        },
        "spec": {
          "containers": [{
            "name": "health",
            "image": "health:latest"
          }]
        },
      },
    }
  }'
```

- The system continuously (every five seconds or less) reconciles actual resources with the configuration. If we were to manually delete one of the pods created by the `ReplicaSet` (or it were to naturally stop), soon a new one would be created to replace it.
- Pods are in the same network and can access each other by name, e.g. in our example, running `curl http://ping:5000` from a `health` container will connect to port 5000 on the ping pod, running `curl http://health:2000` on the ping `ping` will connect

to port 5000 on a health pod chosen by the load balancer.

- Services are exposed on the host machine at their port (so in our example, running `curl http://localhost:2000/health` on the machine running the orchestrator and containers should connect us to a `health` pod's port 5000).
- Thread safety - making many API requests in parallel should never corrupt the program state.
- Tests - the project should contain automated tests that convince you the code does what you meant it to do.

What you're *not* building

- Support for more than one Cluster
- Support for more than one Container in a Pod
- Application-layer (e.g. HTTP) load balancing
- Rolling updates and other deployment patterns (the Deployment object)
- ConfigMaps, Secrets, Persistent Volumes
- Resource Limits and Scheduling
- Health checks and readiness probes
- Autoscaling
- Nodes (support for running Containers on a different machine than the one running the control plane)
- Distributed Control Plane (running control plane components in different processes / nodes, running multiple copies of control plane components)
- Persistent Control Plane state (saving of state to disk or a DB so that it will survive restarting the control plane)

What you're starting from

You are given a POC of an orchestrator (see setup instructions below), which orchestrates Python functions in worker threads instead of Podman containers for simplicity. It should show you one way of doing the control plane logic, but it's not necessarily the only way. Feel free to use it as the basis of your code or choose a completely different architecture for your final submission, but do make sure to get it running before you start developing, because most of the environment issues you could encounter will pop up when you try to run it and it's much easier to debug environment issues when you know you're not mixing them up with bugs in the code. Whether you base your code on the POC or not, any bugs in your submission are your responsibility even if they existed in the POC (and we're not promising that we didn't add a few on purpose).

The POC is in `poc.py`, but we also supply `demo.sh` that shows how to use it.

We will test your code on the VDI environment, so it's a good idea to develop there, but you can choose to develop on another Linux machine **as long as you test the final version on the VDI** (specifically, we strongly advise against developing on Windows or Mac because there's a separate set of weird environment issues there caused by the containers running in a Linux VM that we're not showing you how to deal with). VSCode is [great](#) at developing through SSH.

The VDI environment has some trouble with building and running podman containers. This is mostly because the home directory is by default on network storage and uses a filesystem that doesn't support unix permissions well. You might see an error message about “**overlay**” and “**force_umask**”. A simple way to deal with that is to create a new user with a local home directory. By default, **podman** uses **systemd** to communicate with **cgroupsv2**, and your new user isn't configured for that (you'll get an error message like **The cgroupv2 manager is set to systemd but there is no systemd user session available**), but that can be fixed with one command.

So to get started in the VDI:

Shell

```
# First you'll want to install git
sudo dnf install git-core

# then create a new user
sudo mkdir /var/home
sudo useradd myuser /var/home/myuser
sudo passwd myuser
# now enter the password for the new user

# to switch to the new user:
su myuser
# again, enter the password for the new user

# now you're running as the new user
# fix the systemd issue
loginctl enable-linger `id --user`
# install uv
curl -LsSf https://astral.sh/uv/install.sh | sh
# get the POC
git clone https://github.com/SonOfLilit/actc-hw3.git
# now try running it
```

```
cd actc-hw3/  
uv run poc.py  
  
# and in a separate shell, run the demo  
cd actc-hw3/  
./demo.sh
```

Submission

Submit a file `actc-hw3-ID1-ID2.tar.gz` that contains a directory called `actc-hw3/`.
Submission via the webcourse site. Before submitting, try (in the VDI) running these commands:

```
Shell  
tar xvf actc-hw3-ID1-ID2.tar.gz  
cd actc-hw3/  
pytest  
uv run orchestrator.py
```

How your work will be assessed

Your grade on this submission will be composed of:

- 70% How well you pass our automated test suite
- 30% Our review of your code

We will review your code according to a subset of these aspects:

- Correctness - Does it contain some common bugs? Any other obvious defects?
- Testing - Do your automated tests do a good job of managing defect risk?
- Robustness - Are there dark corners where bugs *could* hide, now or in the future?
- Flexibility - Will it be hard to make the additions we know we'll want to make? How about a few hard to anticipate changes?

This implicitly means the architecture is no less important than the implementation, because it will affect how testable, how robust and how flexible the code is, and if there is nowhere for bugs to hide, there will be no bugs.

You're allowed to use AI for code that runs inside containers you build for testing. You're not allowed to use AI for code that is part of the orchestrator. **It's easy for us to tell if you have**

used AI, and we will do a random check, and we will be forced to give you a 0 mark, so please don't. You are allowed to consult with AI over technical problems you encounter, but try googling the error message first, you'd be surprised at how effective it is!

Assignment Timeline

05/01/2026 - Assignment published

27/01/2026 - (EOD, Israeli time) Submit code

Point of contact

Please email any clarification questions to both aur@sarafconsulting.com AND anastas@technion.ac.il